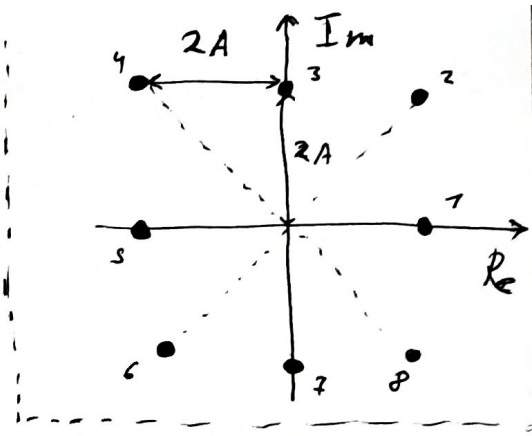
(1) a) $|S_i - S_{i+1}| = 2A \; ; N = 8$

$$P(S_i) = P(S_j) = \frac{1}{N} = \frac{1}{8} \quad \forall i,j$$

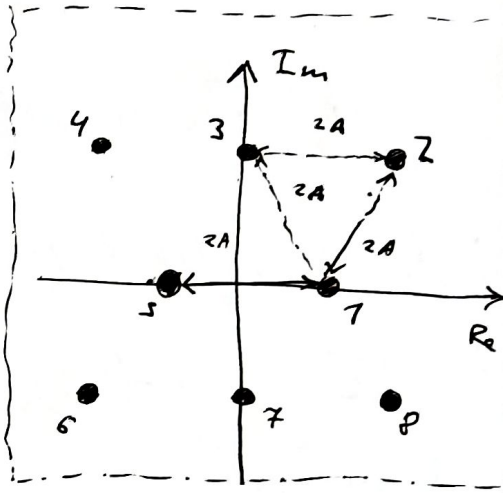
$$\langle \mathcal{E}_s \rangle = \sum_{i=1}^{N} |S_i|^2 P(S_i) = \frac{1}{8}\left(\sum_{i=1}^{4} |S_{2i}|^2 + \sum_{i=0}^{3} |S_{2i+1}|^2 \right) = \frac{1}{8}\left(4 \cdot 8A^2 + 4 \cdot 4A^2\right) =$$

$$= A^2 6$$



b) $$\langle \mathcal{E}_s \rangle = \frac{1}{8} \cdot \sum_{i=1}^{8} |S_i|^2 = \frac{1}{8}\Big((|S_1|^2 + |S_5|^2) + (|S_3|^2 + |S_7|^2) + (|S_2|^2 + |S_4|^2 + |S_6|^2 + |S_8|^2) \Big) =$$

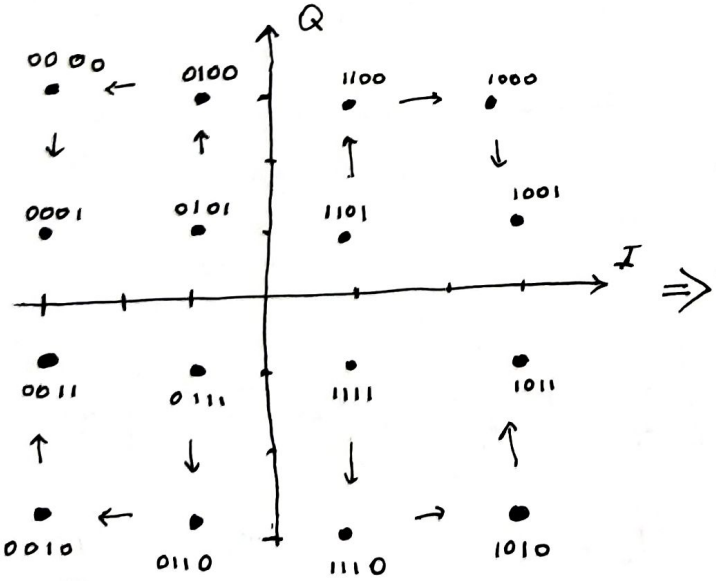
$$= \frac{1}{8}\left(2 \cdot A^2 + 2 \cdot 3A^2 + 4 \cdot 7A^2\right) = 4.5\, A^2$$



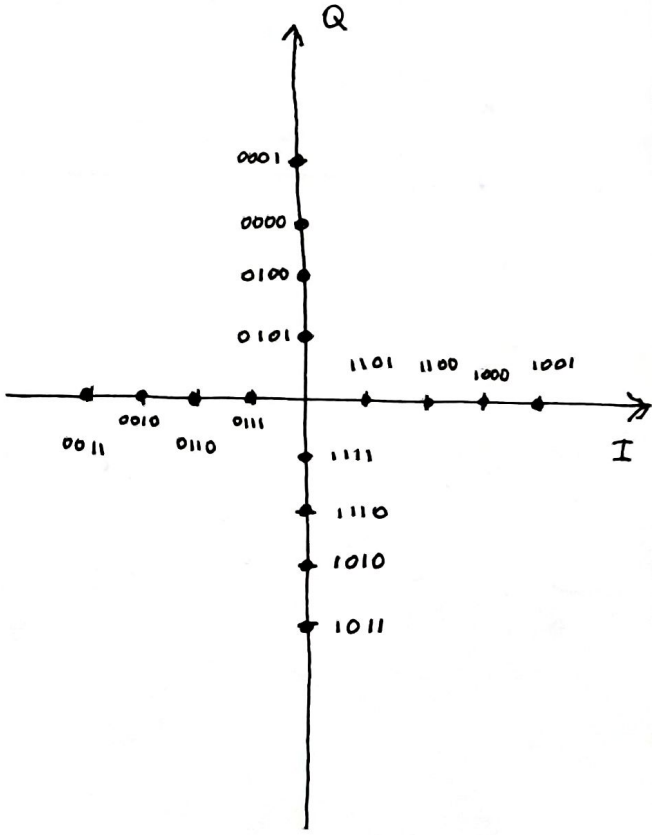
constellation (b) is more power-efficient than constellation (a)

---

(2) We know, QAM 16:



$\Rightarrow$



③ Frequency-modulated signal with memory.
Continuous phase with partial response CPM.

$h = \frac{1}{2}$; $g(t) = \begin{cases} \frac{1}{4T} , & 0 \le t \le 2T \\ 0 , & \text{otherwise} \end{cases}$

Passband signal : $s(t) = \sqrt{\frac{2E}{T}} \cos(2\pi f_c t + \phi(t;\vec{I}) + \phi_0)$

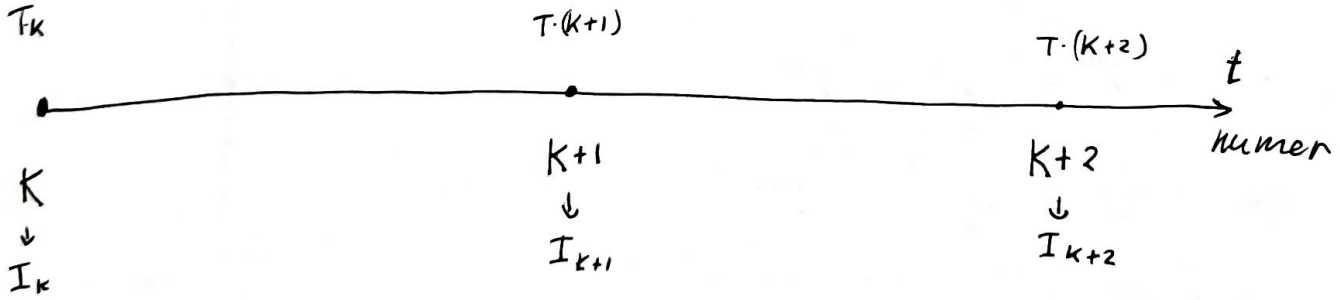
- $\phi(t;\vec{I}) = 4\pi T f_d \int_{-\infty}^{t} \sum_{k=-\infty}^{n} I_k g(\tau - kT)\, d\tau = \left\{ \begin{array}{l} q(t) = \int_0^t g(\tau)\, d\tau \\ h = 2 f_d T \end{array} \right.$

$= 2\pi h \sum_{k=-\infty}^{n} I_k q(t-kT)$, $nT \le t \le (n+1)T$

- $\{I_k\}$; $I_k \in \{1; -1\}$ ← WLOG For plotting phase-trellis
- $I_k \in \{-(M-1), \dots, -1, 1, 2, \dots, M-1\}$

I current stage number = k, Accumulated phase = ϕ_0.

$\phi(t;\vec{I}) = \pi \sum_{k=0}^{n} I_k (q(t-kT))$

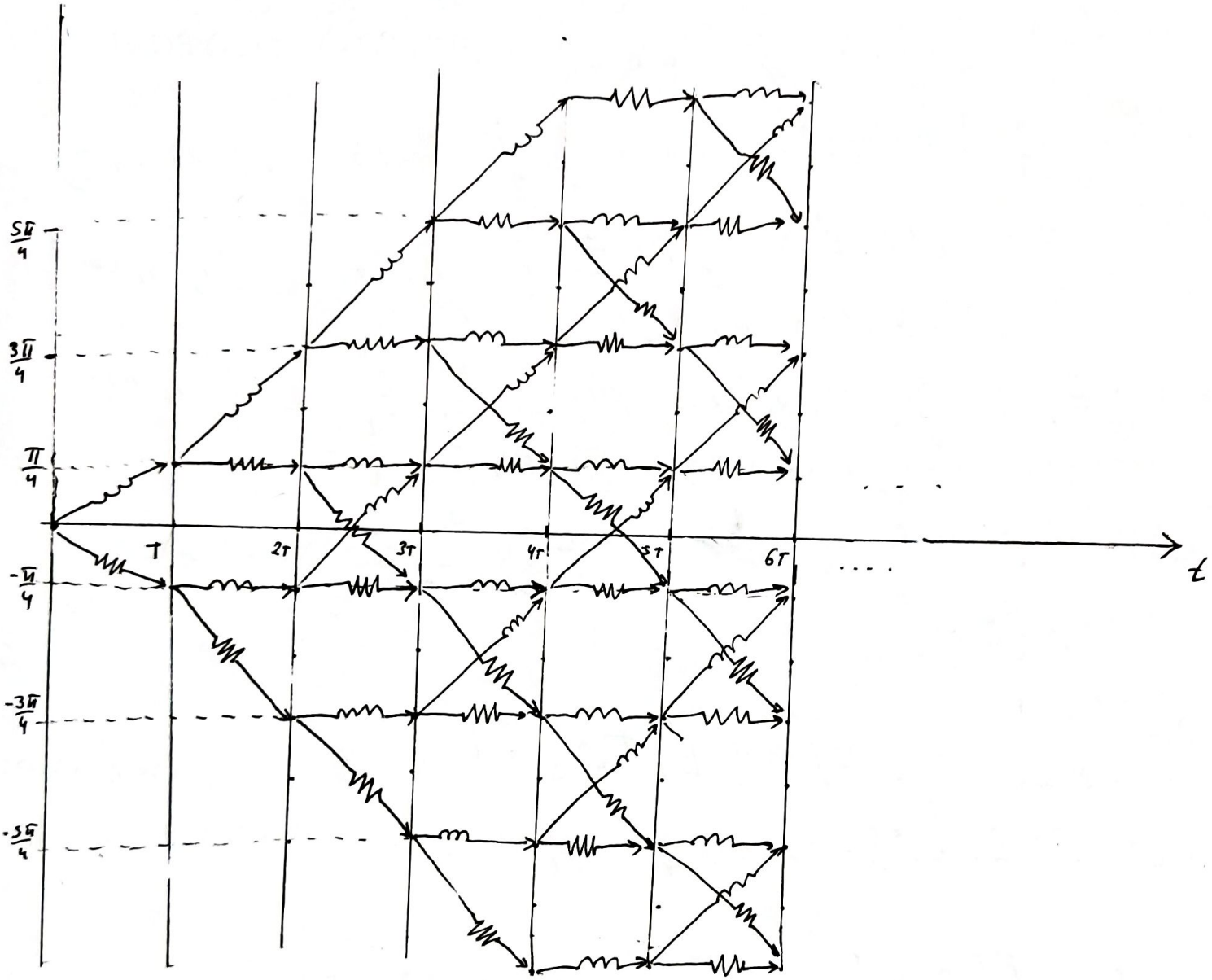


$\phi_k = \phi_0$ $\quad \phi_{k+1} = \phi_0 + \pi\left[\frac{I_{k-1}}{4} + \frac{I_k}{4}\right]$ $\quad \phi_{k+2} = \phi_{k+1} + \pi\left[\frac{I_k}{4} + \frac{I_{k+1}}{4}\right]$

$\phi_2 = \phi_{2-1} + \pi\left[\frac{I_{2-2}}{4} + \frac{I_{2-1}}{4}\right]$, $\phi_0 = 0$.

· phase-trellis $I_k \in \{1; -1\}$

⟶ $\Rightarrow I_k = 1$; ⟿ $I_k = -1$.



(4) Phase-modulated signal

$u(t) = \sum_n I_n g(t - nT)$; $I_n \in \left\{ \frac{1+i}{\sqrt{2}}, \frac{1-i}{\sqrt{2}}, \frac{-1+i}{\sqrt{2}}, \frac{-1-i}{\sqrt{2}} \right\}$

$P(I_n) = P(I_k) = \frac{1}{4} \quad \forall n, k \in \mathbb{N}$; $P(I_n \cap I_k) = P(I_n) P(I_k)$

1) $S_u(f) = \frac{1}{T} |G(f)|^2 S_I(f) = \frac{1}{T} \sum_{k=-\infty}^{+\infty} G_k(f) \exp(-i 2\pi k f T)$

$G_k(f) = E[S_e(f; \vec{I}_k) S_e^*(f; \vec{I}_0)]$, $S_e(f; \vec{I}_k) = F[S_e(t; \vec{I}_k)]$

$S_e(t; \vec{I}_k) = I_k g(t - kT)$

$G_k(f) = E[I_k \cdot I_0^* F[g(t-kT)] F^*[g(t-kT)]] = E[I_k I_0^*] \cdot G(f) \cdot$

$\cdot \exp(-i 2\pi f kT) \cdot G^*(f) \exp(i 2\pi f kT) = R_I(k) |G(f)|^2 \Rightarrow$

$\Rightarrow S_u(f) = \frac{1}{T} \sum_{k=-\infty}^{+\infty} G_k(f) \exp(-i 2\pi k f T) = \frac{|G(f)|^2}{T} \underbrace{\sum_{k=-\infty}^{+\infty} R_I(k) \exp(-i 2\pi k f T)}_{S_I(f)}$

$G(f) = F[g(t)]$

---

2) $S_I(f) - ?$

$R_I(k) = E[I_k \cdot I_0^*] = E[I_k] \cdot E[I_0^*] = 0$, $k \neq 0$ $\Rightarrow R_I(k) = \delta(k) \Rightarrow$

$R_I(0) = E[I_0 I_0^*] = \frac{1}{4} \cdot 1 + \frac{1}{4} \cdot 1 + \frac{1}{4} \cdot 1 + \frac{1}{4} \cdot 1 = 1 \Rightarrow S_I(f) = 1$

---

3) $G_a(f) = F[g_a(t)]$; $g_a(t) = \begin{cases} A, & 0 \le t \le T \\ 0, & \text{otherwise} \end{cases}$ $\Rightarrow$

$G_a(f) = \int_0^T A e^{-i 2\pi f t} dt = T A e^{-\pi i f T} \operatorname{sinc}(\pi f T) \Rightarrow$

$\Rightarrow S_{u_a}(f) = \frac{(TA)^2 \operatorname{sinc}^2(\pi f T)}{T} = T A^2 \operatorname{sinc}^2(\pi f T)$

$$G_b(f) = F[g_b(t)]; \quad g_b(t) = \begin{cases} A \sin\left(\frac{\pi t}{T}\right), & 0 \le t \le T \\ 0, & \text{otherwise} \end{cases}$$

$$G_b(f) = \int_0^T A \frac{\left(e^{i\frac{\pi}{T}t} - e^{-i\frac{\pi}{T}t}\right)}{2i} e^{-2\pi i f t} dt =$$

$$\int_0^T e^{it\left(\frac{\pi}{T} - 2\pi f\right)} dt = T e^{i\frac{T}{2}\left(\frac{\pi}{T} - 2\pi f\right)} \cdot \text{sinc}\left(\frac{T}{2}\left(\frac{\pi}{T} - 2\pi f\right)\right) \Rightarrow$$

$$-\int_0^T e^{it\left(-\frac{\pi}{T} - 2\pi f\right)} dt = -T e^{-i\frac{T}{2}\left(\frac{\pi}{T} + 2\pi f\right)} \cdot \text{sinc}\left(\frac{T}{2}\left(\frac{\pi}{T} + 2\pi f\right)\right)$$

$$\Rightarrow G_b(f) = \frac{AT}{2i} \cdot e^{-i\pi T f} \cdot i\left[\text{sinc}\left(\frac{\pi}{2} - \pi T f\right) + \text{sinc}\left(\frac{\pi}{2} + \pi T f\right)\right] =$$

$$= \frac{AT}{2} e^{-i\pi T f}\left[\text{sinc}\left(\frac{\pi}{2} - \pi T f\right) + \text{sinc}\left(\frac{\pi}{2} + \pi T f\right)\right]$$

$$S_{u_b}(f) = \frac{A^2 T}{4}\left[\text{sinc}\left(\frac{\pi}{2} - \pi T f\right) + \text{sinc}\left(\frac{\pi}{2} + \pi T f\right)\right]^2.$$

4) The PSD of S_{u_b} decays faster than the PSD of S_{u_a}. As a result, out-of-band emissions are lower, but the main lobe becomes wider.

1 $S_a = TA^2 \cdot S(\pi T x)^2$

2 $S_b = \frac{TA^2}{4} \cdot \left(S\left(\frac{\pi}{2} - \pi T x\right) + S\left(\frac{\pi}{2} + \pi T x\right)\right)^2$

3 $T = 0.9$

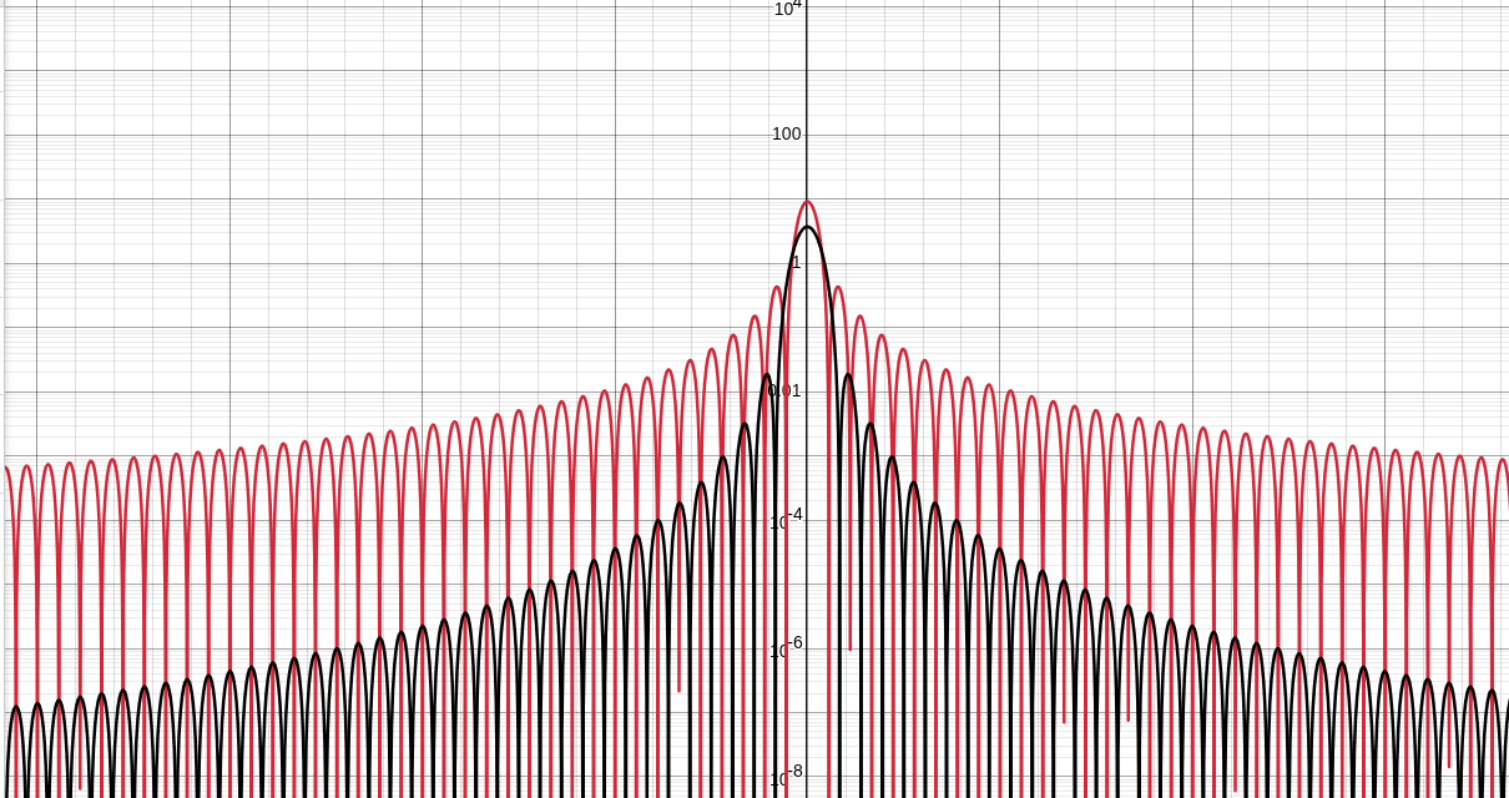
−10 10

4 $A = 3.2$

−10 10

5 $S(a) = \frac{\sin(a)}{a}$

6



```python
import numpy as np
import matplotlib.pyplot as plt
import random

from scipy.fft import fft, ifft, fftfreq, fftshift
from scipy.signal import welch

from QAM_mapper import QAM_mapper
from MSK_mapper import MSK_mapper
from plotting_funcs import plot_constellation

PI = np.pi

def occupied_bandwidth_99(x, fs, threshold=0.99):

    N = len(x)
    X = fft(x)

    P = (np.abs(X)/N)**2

    f = fftfreq(N, 1/fs)

    order = np.argsort(np.abs(f))
    f_sorted = np.abs(f)[order]
    P_sorted = P[order]

    df = fs / N

    cum_power = np.cumsum(P_sorted) * df
    total_power = np.sum(P) * df

    idx = np.searchsorted(cum_power, threshold * total_power)
    f_edge = f_sorted[idx]

    return f_edge



## Model parameters

T_signal = 1e-2
dt = 1e-5
df = 1/dt
numb_points = int(T_signal / dt)
t = np.linspace(0, T_signal, num=numb_points, endpoint=False)
Energy_of_impulse = 1
h = 0.5


bit_rate = 1e4
numb_bits = int(T_signal * bit_rate * np.log2(16) * np.log2(64) * np.log2(256))
bit_sequence = [random.randint(0, 1) for _ in range(numb_bits)]



## Create objects
qam4 = QAM_mapper(4, bit_rate, Energy_of_impulse)
qam16 = QAM_mapper(16, bit_rate, Energy_of_impulse)
qam64 = QAM_mapper(64, bit_rate, Energy_of_impulse)
qam256 = QAM_mapper(256, bit_rate, Energy_of_impulse)

msk = MSK_mapper(bit_rate, Energy_of_impulse, h)


## Find baseband signals
baseband_qam4_signal = qam4.get_baseband_signal(bit_sequence)
baseband_qam16_signal = qam16.get_baseband_signal(bit_sequence)
baseband_qam64_signal = qam64.get_baseband_signal(bit_sequence)
baseband_qam256_signal = qam256.get_baseband_signal(bit_sequence)

baseband_qam4_signal_t = baseband_qam4_signal(t)
baseband_qam16_signal_t = baseband_qam16_signal(t)
baseband_qam64_signal_t = baseband_qam64_signal(t)
baseband_qam256_signal_t = baseband_qam256_signal(t)

baseband_msk_signal_t, t_msk, msk_phases = msk.get_modulate_sequence_and_time_and_phase(0, T_signal, numb_points, bit_sequence)

if np.array_equal(t, t_msk): print("ploho")

## Find spectrums
N = np.size(baseband_qam4_signal_t)
freqs = fftfreq(N, dt)

qam4_spec = fft(baseband_qam4_signal_t) / N
qam16_spec = fft(baseband_qam16_signal_t) / N
qam64_spec = fft(baseband_qam64_signal_t) / N
qam256_spec = fft(baseband_qam256_signal_t) / N

msk_spec = fft(baseband_msk_signal_t) / N


## Find teoretic spectrum effectivity

# find bandwidth where 99% power of all spector's power
qam4_bandwidth = 2*occupied_bandwidth_99(baseband_qam4_signal_t, df)
qam16_bandwidth = 2*occupied_bandwidth_99(baseband_qam16_signal_t, df)
qam64_bandwidth = 2*occupied_bandwidth_99(baseband_qam64_signal_t, df)
qam256_bandwidth = 2*occupied_bandwidth_99(baseband_qam256_signal_t, df)

msk_bandwidth = 2*occupied_bandwidth_99(baseband_msk_signal_t, df)

qam4_spec_eff = bit_rate / qam4_bandwidth
qam16_spec_eff = bit_rate / qam16_bandwidth
qam64_spec_eff = bit_rate / qam64_bandwidth
qam256_spec_eff = bit_rate / qam256_bandwidth

msk_spec_eff = bit_rate / msk_bandwidth


qam4_spec_eff_teor = np.log2(4)
qam16_spec_eff_teor = np.log2(16)
qam64_spec_eff_teor = np.log2(64)
qam256_spec_eff_teor = np.log2(256)

msk_spec_eff_teor = 1 / 1  # bit_rate_msk / bandwidth

print("Bandwidth efficientivity:")
print("BW_E QAM4=", qam4_spec_eff, "  BW_E QAM4 teoreical=", qam4_spec_eff_teor)
print("BW_E QAM16=", qam16_spec_eff, "  BW_E QAM16 teoreical=", qam16_spec_eff_teor)
print("BW_E QAM64=", qam64_spec_eff, "  BW_E QAM64 teoreical=", qam64_spec_eff_teor)
print("BW_E QAM256=", qam256_spec_eff, "  BW_E QAM256 teoreical=", qam256_spec_eff_teor)

print("BW_E MSK=", msk_spec_eff, "  BW_E MSK teoreical=", msk_spec_eff_teor)
```

```
## Find PSD params
freq_psd_qam4, psd_qam4 = welch(baseband_qam4_signal_t, df, window='hann', nperseg=1024, scaling='density')
freq_psd_qam16, psd_qam16 = welch(baseband_qam16_signal_t, df, window='hann', nperseg=1024, scaling='density')
freq_psd_qam64, psd_qam64 = welch(baseband_qam64_signal_t, df, window='hann', nperseg=1024, scaling='density')
freq_psd_qam256, psd_qam256 = welch(baseband_qam256_signal_t, df, window='hann', nperseg=1024, scaling='density')

freq_psd_msk, psd_msk = welch(baseband_msk_signal_t, df, window='hann', nperseg=1024, scaling='density')


## Plotting signals's Amp and Phases

# plotting qam
fig1, axs = plt.subplots(2, 1, figsize=(10, 8))

axs[0].plot(t, np.abs(baseband_qam4_signal_t),            '-', label='qam 4')
axs[0].plot(t, np.abs(baseband_qam16_signal_t),              '-', label='qam 16')
axs[0].plot(t, np.abs(baseband_qam64_signal_t),            '-', label='qam 64')
axs[0].plot(t, np.abs(baseband_qam256_signal_t),             '-', label='qam 256')
axs[0].set_title('Amplitudes of QAM')
axs[0].set_xlabel('time sec')
axs[0].set_ylabel('Amp')
axs[0].legend()
axs[0].grid(True)

axs[1].plot(t, np.angle(baseband_qam4_signal_t),            '-', label='qam 4')
axs[1].plot(t, np.angle(baseband_qam16_signal_t),              '-', label='qam 16')
axs[1].plot(t, np.angle(baseband_qam64_signal_t),            '-', label='qam 64')
axs[1].plot(t, np.angle(baseband_qam256_signal_t),             '-', label='qam 256')
axs[1].set_title('Phases of QAM')
axs[1].set_xlabel('time sec')
axs[1].set_ylabel('Radians')
axs[1].legend()
axs[1].grid(True)

plt.tight_layout()
plt.show()

# ploting msk
fig2, axs = plt.subplots(2, 1, figsize=(10, 8))

x_vals = np.arange(0, T_signal, msk._T_symb)
axs[1].vlines(x_vals, np.min(msk_phases), np.max(msk_phases), linestyles='dashed')

axs[0].plot(t_msk, np.real(baseband_msk_signal_t),             '-', label='msk real')
axs[0].plot(t_msk, np.imag(baseband_msk_signal_t),             '-', label='msk imag')
axs[0].set_title('Baseband amplitudes of MSK')
axs[0].set_xlabel('time sec')
axs[0].set_ylabel('Amp')
axs[0].legend()
axs[0].grid(True)

axs[1].plot(t_msk, msk_phases,             '-', label='msk')
axs[1].set_title('BasebansPhases of MSK')
axs[1].set_xlabel('time sec')
axs[1].set_ylabel('Radians')
axs[1].legend()
axs[1].grid(True)

plt.tight_layout()
plt.show()
```

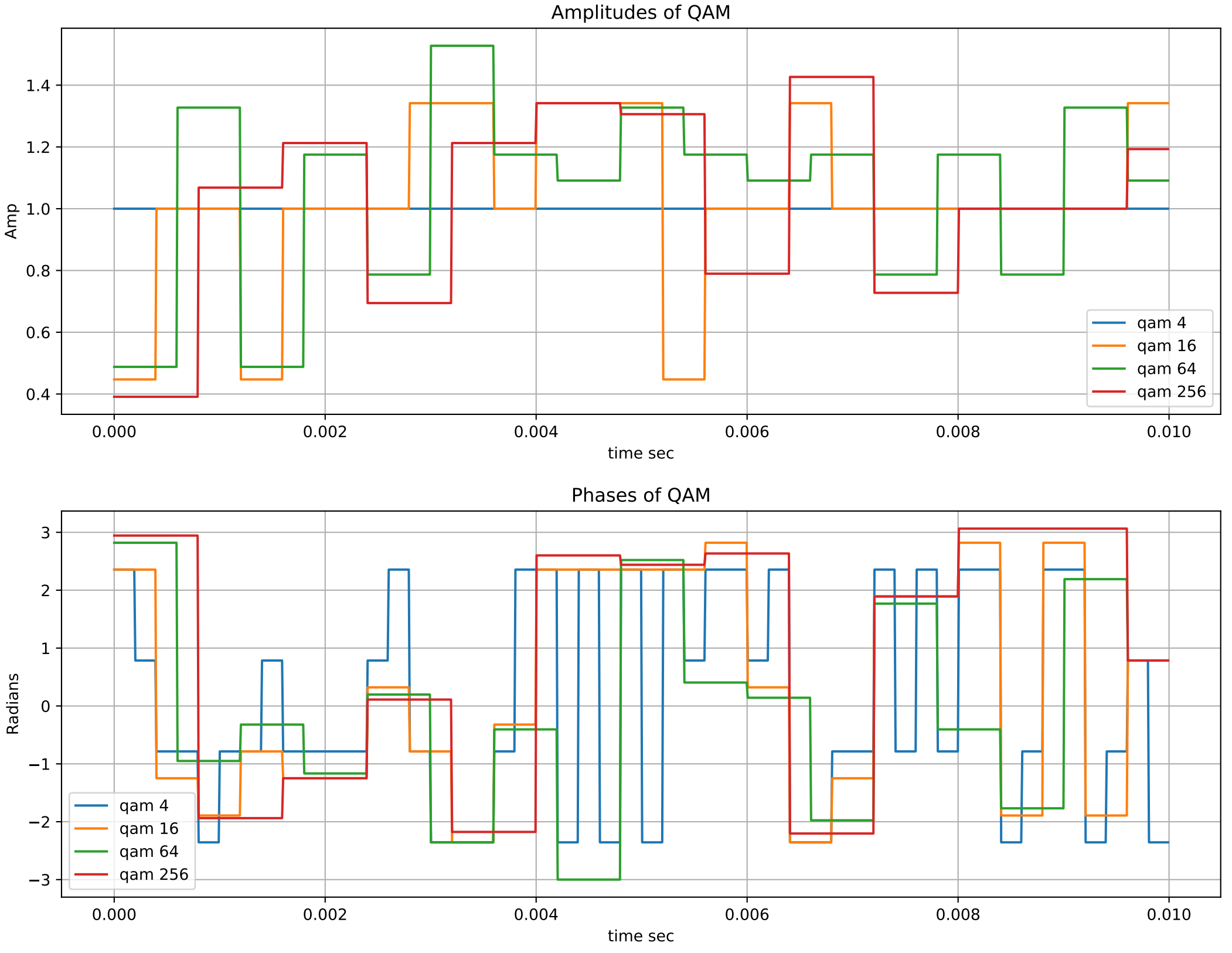
```
## Plotting spectrums
# plotting qam
fig3, axs = plt.subplots(4, 1, figsize=(16, 8))

axs[0].axvline(qam4_bandwidth/2, color='red', linestyle='--', label='right spectral edge')
axs[0].axvline(-qam4_bandwidth/2, color='red', linestyle='--', label='left spectral edge')
axs[0].plot(freqs, np.abs(qam4_spec), '--')
axs[0].set_title('Amplitudes of QAM4')
axs[0].set_xlabel('freq Hz')
axs[0].set_ylabel('Amp')
axs[0].legend()
axs[0].grid(True)

axs[1].axvline(qam16_bandwidth/2, color='red', linestyle='--', label='right spectral edge')
axs[1].axvline(-qam16_bandwidth/2, color='red', linestyle='--', label='left spectral edge')
axs[1].plot(freqs, np.abs(qam16_spec), '--')
axs[1].set_title('Amplitudes of QAM16')
axs[1].set_xlabel('freq Hz')
axs[1].set_ylabel('Amp')
axs[1].legend()
axs[1].grid(True)

axs[2].axvline(qam64_bandwidth/2, color='red', linestyle='--', label='right spectral edge')
axs[2].axvline(-qam64_bandwidth/2, color='red', linestyle='--', label='left spectral edge')
axs[2].plot(freqs, np.abs(qam64_spec), '--')
axs[2].set_title('Amplitudes of QAM64')
axs[2].set_xlabel('freq Hz')
axs[2].set_ylabel('Amp')
axs[2].legend()
axs[2].grid(True)

axs[3].axvline(qam256_bandwidth/2, color='red', linestyle='--', label='right spectral edge')
axs[3].axvline(-qam256_bandwidth/2, color='red', linestyle='--', label='left spectral edge')
axs[3].plot(freqs, np.abs(qam256_spec), '--')
axs[3].set_title('Amplitudes of QAM256')
axs[3].set_xlabel('freq Hz')
axs[3].set_ylabel('Amp')
axs[3].legend()
axs[3].grid(True)

plt.tight_layout()
plt.show()

# ploting msk
fig4, axs = plt.subplots(2, 1, figsize=(10, 8))

axs[0].axvline(msk_bandwidth/2, color='red', linestyle='--', label='right spectral edge')
axs[0].axvline(-msk_bandwidth/2, color='red', linestyle='--', label='left spectral edge')
axs[0].plot(freqs, np.abs(msk_spec), '--')
axs[0].set_title('MSK spectrum\'s amplitude')
axs[0].set_xlabel('freq Hz')
axs[0].set_ylabel('Amp')
axs[0].legend()
axs[0].grid(True)

axs[1].plot(freqs, np.angle(msk_spec), '--')
axs[1].set_title('MSK spectrum\'s phase')
axs[1].set_xlabel('freq Hz')
axs[1].set_ylabel('Radians')
```

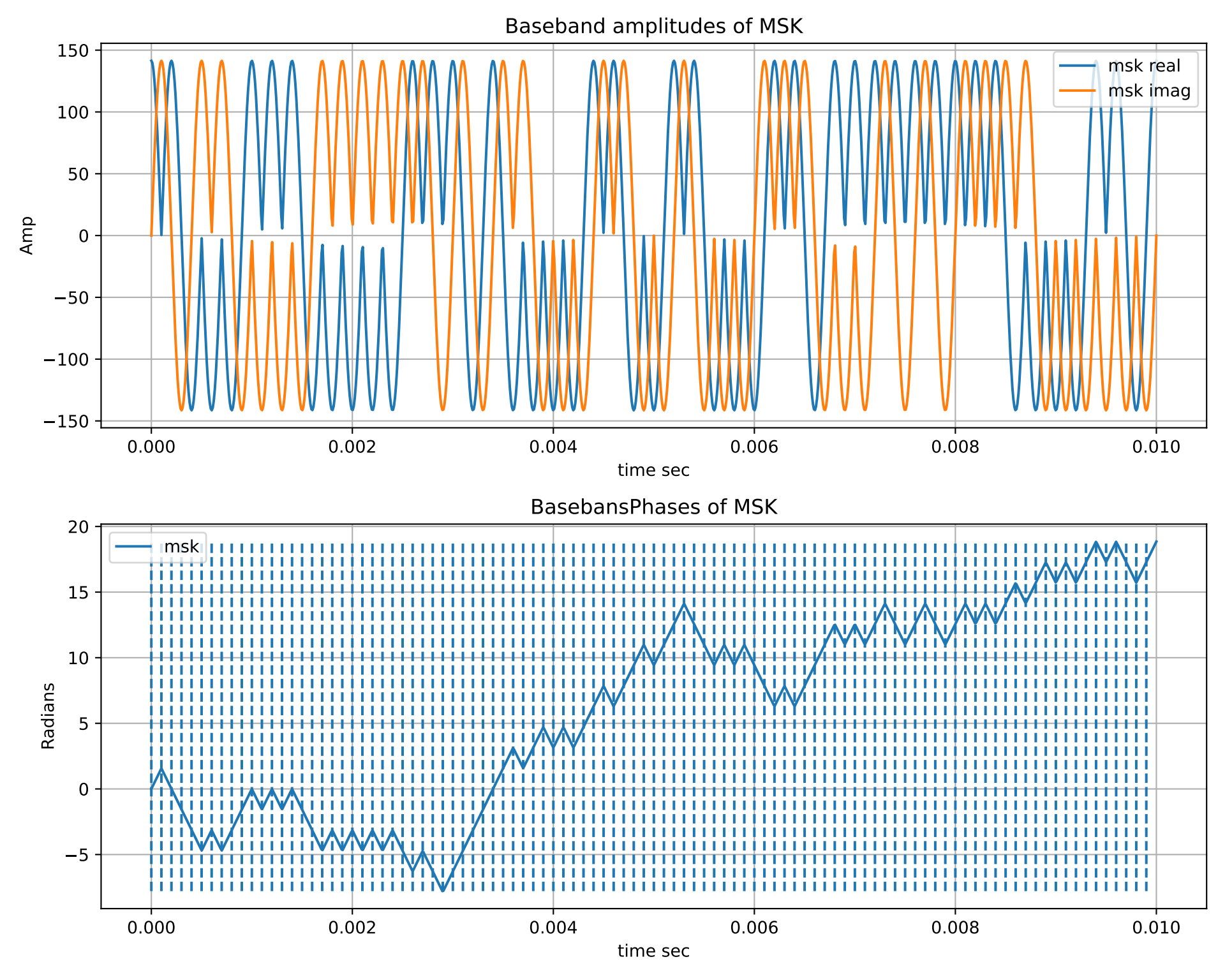
```
axs[1].grid(True)

plt.tight_layout()
plt.show()


## Plotting PSD
# QAM4
fig5, axs = plt.subplots(1, 1, figsize=(10, 8))

axs.semilogy(freq_psd_qam4, psd_qam4, '--', label='QAM 4')
axs.semilogy(freq_psd_msk, psd_msk , '--', label='MSK')
axs.set_title('PSD QAM4 and MSK')
axs.legend()
axs.set_xlabel('freq Hz')
axs.set_ylabel('PSD')
axs.grid(True)

plt.tight_layout()
plt.show()

# QAM16
fig6, axs = plt.subplots(1, 1, figsize=(10, 8))

axs.semilogy(freq_psd_qam16, psd_qam16, '--', label='QAM 16')
axs.semilogy(freq_psd_msk, psd_msk , '--', label='MSK')
axs.set_title('PSD QAM16 and MSK')
axs.legend()
axs.set_xlabel('freq Hz')
axs.set_ylabel('PSD')
axs.grid(True)

plt.tight_layout()
plt.show()

# QAM64
fig7, axs = plt.subplots(1, 1, figsize=(10, 8))

axs.semilogy(freq_psd_qam64, psd_qam64, '--', label='QAM 64')
axs.semilogy(freq_psd_msk, psd_msk , '--', label='MSK')
axs.set_title('PSD QAM64 and MSK')
axs.legend()
axs.set_xlabel('freq Hz')
axs.set_ylabel('PSD')
axs.grid(True)

plt.tight_layout()
plt.show()

# QAM256
fig8, axs = plt.subplots(1, 1, figsize=(10, 8))

axs.semilogy(freq_psd_qam256, psd_qam256, '--', label='QAM 256')
axs.semilogy(freq_psd_msk, psd_msk , '--', label='MSK')
axs.set_title('PSD QAM256 and MSK')
axs.legend()
axs.set_xlabel('freq Hz')
axs.set_ylabel('PSD')
axs.grid(True)

plt.tight_layout()
plt.show()
```

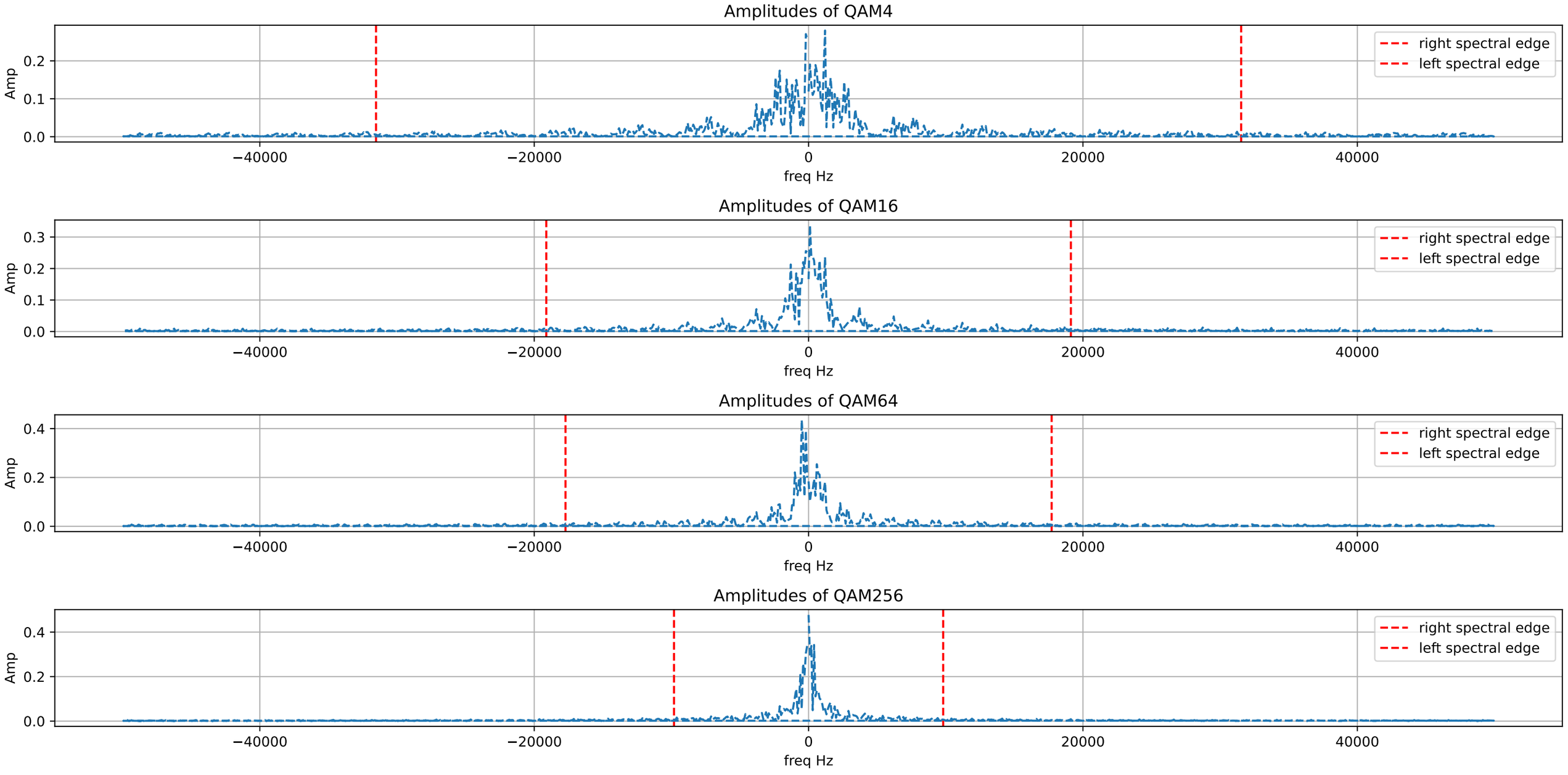
```python
import numpy as np


def int_to_bits_lsb(value: int, length: int) -> np.ndarray:

    if value < 0:
        raise ValueError("value >= 0 must be")
    if value >= (1 << length):
        raise ValueError("value size not combinian into import length")

    return np.array([(value >> i) & 1 for i in range(length)],
                    dtype=np.uint8)


def bits_to_int_lsb(bits):
    value = 0

    for i, bit in enumerate(bits):
        value |= int(bit) << i

    return value



class QAM_mapper:

    def __init__(self, M, bit_rate, E_symb_avg=1):

        self._M                     = M
        self._bit_rate              = bit_rate
        self._num_bits_in_qam       = int(np.log2(M))
        self._symbol_rate           = self._bit_rate/self._num_bits_in_qam
        self._T_symb                = 1/self._symbol_rate
        self._E_symb_avg            = E_symb_avg
        self._E_bit_avg             = E_symb_avg/np.log2(M)
        self._mod_signals           = self._get_mod_arr()



    def get_QAM4_signals_arr(self):
        arr_signals = np.zeros(4, dtype=complex)

        for numb in range(0, 4):
            bits = int_to_bits_lsb(numb, 2)
            arr_signals[numb] = (1/np.sqrt(2))*((1 - 2*bits[0]) + 1j*(1 - 2*bits[1]))

        return arr_signals * np.sqrt(self._E_symb_avg)


    def get_QAM16_signals_arr(self):
        arr_signals = np.zeros(16, dtype=complex)

        for numb in range(0, 16):
            bits = int_to_bits_lsb(numb, 4)
            arr_signals[numb] = (1/np.sqrt(10))*((1 - 2*bits[0])*(2 - (1 - 2*bits[2])) + 1j*(1 - 2*bits[1])*(2 - (1 - 2*bits[3])))

        return arr_signals * np.sqrt(self._E_symb_avg)


    def get_QAM64_signals_arr(self):
        arr_signals = np.zeros(64, dtype=complex)

        for numb in range(0, 64):
            bits = int_to_bits_lsb(numb, 6)
            arr_signals[numb] = (1/np.sqrt(42))*(  (1 - 2*bits[0])*(4 -(1 - 2*bits[2])*(2 - (1 - 2*bits[4]))) + 1j*(1 - 2*bits[1])*(4 -(1 - 2*bits[3])*(2 - (1 - 2*bits

        return arr_signals * np.sqrt(self._E_symb_avg)


    def get_QAM256_signals_arr(self):
        arr_signals = np.zeros(256, dtype=complex)

        for numb in range(0, 256):
            bits = int_to_bits_lsb(numb, 8)
            arr_signals[numb] = (1/np.sqrt(170))*(  (1 - 2*bits[0])*( 8 -(1 - 2*bits[2])*( 4 -(1 - 2*bits[4])*(2 - (1 - 2*bits[6])) ) )  +  1j*(1 - 2*bits[1]) * ( 8 -


        return arr_signals * np.sqrt(self._E_symb_avg)


    def _get_mod_arr(self):

        match self._M:
            case 4:
                return self.get_QAM4_signals_arr()
            case 16:
                return self.get_QAM16_signals_arr()
            case 64:
                return self.get_QAM64_signals_arr()
            case 256:
                return self.get_QAM256_signals_arr()
            case _:
                raise ValueError("There aren't modulation M=" + str(self._M))

        return 0


    def get_average_energy(self):

        return sum(x * x for x in self._mod_signals) / self._M


    def get_baseband_signal(self, bit_sequence):

        if ((np.size(bit_sequence) % self._num_bits_in_qam) != 0): raise ValueError("the number of bits is not a multiple of the number of bits per symbo")

        bit_seq_sz = np.size(bit_sequence)

        def baseband_signal_t(t):

            n = t // self._T_symb
            start_bits = int(n*self._num_bits_in_qam)
            end_bits = int((n+1)*self._num_bits_in_qam)

            if start_bits >= bit_seq_sz or end_bits >= bit_seq_sz: raise ValueError("There aren't bits for do modulating")

            cur_num = bits_to_int_lsb(bit_sequence[start_bits:end_bits])

            return self._mod_signals[cur_num]

        def baseband_signal(t_arr):
            if np.isscalar(t_arr):
                return baseband_signal_t(t_arr)
```

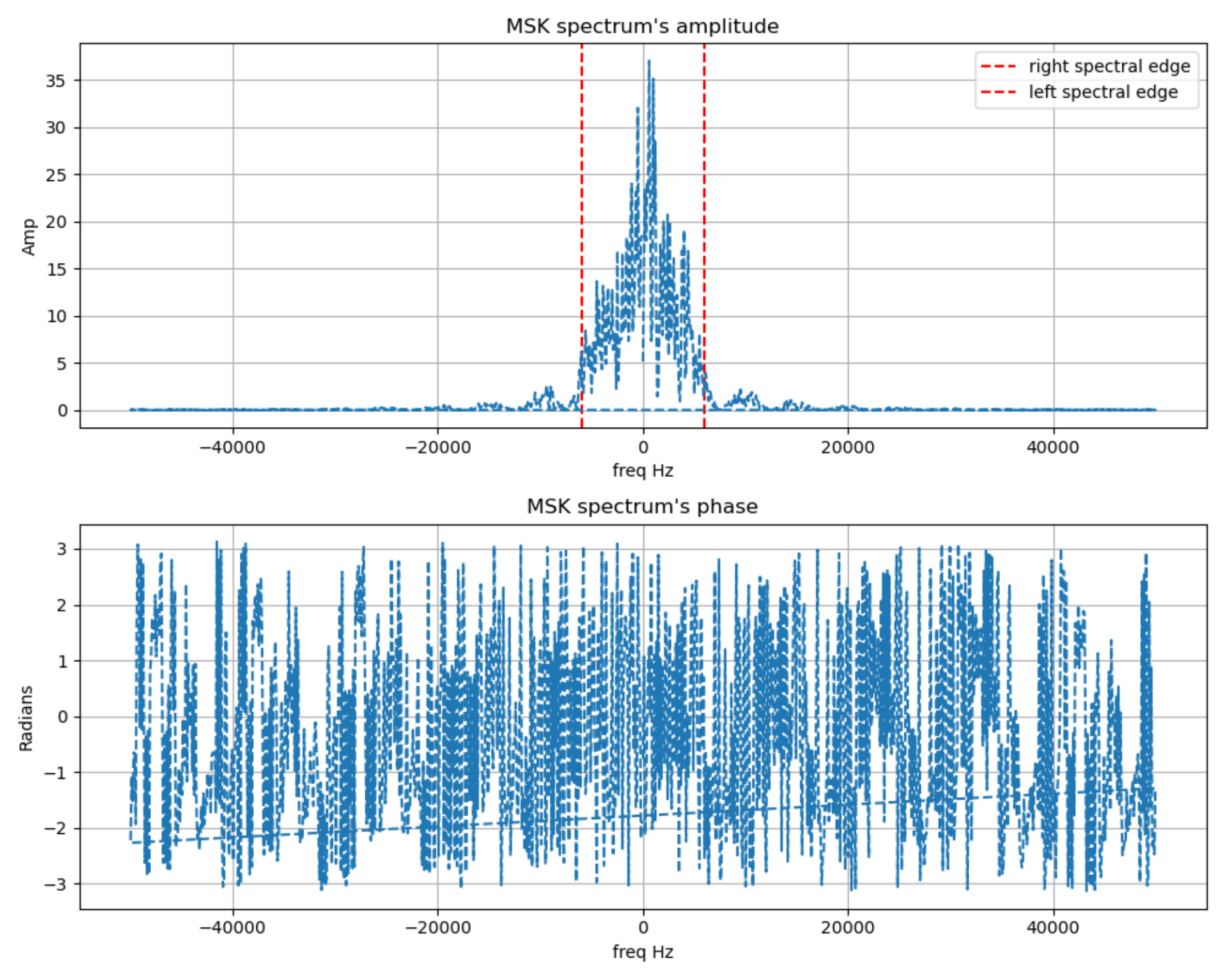
```
        result = np.zeros_like(t_arr, dtype=complex)

        for i in range(0, np.size(t_arr)):
            result[i] = baseband_signal_t(t_arr[i])

        return result

    return baseband_signal


def get_modulate_sequence_and_time(self, t_start, t_end, numb_of_points, bit_sequence):

    if (t_end - t_start) <= 0:  raise ValueError("t_end <= t_start")
    if ((t_end - t_start) // self._T_symb + 1) * self._num_bits_in_qam > np.size(bit_sequence): raise ValueError("PËnsufficient bits to form the baseband signal")
    if numb_of_points <= 0: raise ValueError("numb_points must be > 0")

    t = np.linspace(t_start, t_end, numb_of_points)
    result = np.zeros(numb_of_points, dtype=complex)

    for i in range(0, numb_of_points):
        n = (t[i] - t[0]) // self._T_symb

        start_bits = int(n*self._num_bits_in_qam)
        end_bits = int((n+1)*self._num_bits_in_qam)

        cur_signal_val = bits_to_int_lsb(bit_sequence[start_bits:end_bits])

        result[i] = self._mod_signals[cur_signal_val]

    return result, t
```

```
import numpy as np


PI = np.pi

def int_to_bits_lsb(value: int, length: int) -> np.ndarray:

    if value < 0:
        raise ValueError("value >= 0 must be")
    if value >= (1 << length):
        raise ValueError("value size not combinian into import length")

    return np.array([(value >> i) & 1 for i in range(length)],
                    dtype=np.uint8)



class MSK_mapper:

    def __init__(self, bit_rate, signal_energy=1, h=0.5):

        self._h             = h
        self._bit_rate      = bit_rate
        self._T_symb        = 1/self._bit_rate
        self._E             = signal_energy


    def _get_theta_n(self, bit_sequence, n):

        if (n >= np.size(bit_sequence)): raise ValueError("n >= size of bit sequence")

        if n == 0: return 0

        theta = 0

        for i in range(0, n):
            theta += (2*bit_sequence[i] - 1)

        theta_n = PI*self._h*theta

        return theta_n


    def get_average_energy(self):

        return self._E


    def get_baseband_signal(self, bit_sequence):

        def baseband_signal_t(t):
            n = int(t // self._T_symb)
            theta_n = self._get_theta_n(bit_sequence, n)

            phi_t = theta_n + (PI*self._h*(2*bit_sequence[n] - 1)) * (t - n*self._T_symb)/self._T_symb


            return np.sqrt(2*self._E/self._T_symb)*np.exp(1j*phi_t)

        def baseband_signal(t_arr):

            if np.isscalar(t_arr):
                return baseband_signal_t(t_arr)

            result = np.zeros_like(t_arr, dtype=complex)

            for i in range(0, np.size(t_arr)):
                result[i] = baseband_signal_t(t_arr[i])

            return result

        return baseband_signal


    def get_modulate_sequence_and_time_and_phase(self, t_start, t_end, numb_of_points, bit_sequence):

        if (t_end - t_start) <= 0:  raise ValueError("t_end <= t_start")
        if ((t_end - t_start) // self._T_symb + 1)  > np.size(bit_sequence): raise ValueError("PËnsufficient bits to form the baseband signal")
        if numb_of_points <= 0: raise ValueError("numb_points must be > 0")

        t = np.linspace(t_start, t_end, numb_of_points)
        result = np.zeros(numb_of_points, dtype=complex)
        phases = np.zeros(numb_of_points, dtype=complex)

        theta_n = 0
        cur_n = 0

        for i in range(0, numb_of_points):
            n = int((t[i] - t[0]) // self._T_symb)

            if cur_n != n:
                theta_n += (2*bit_sequence[n - 1] - 1)*PI*self._h
                cur_n = n

            phi_t = theta_n + (PI*self._h*(2*(bit_sequence[n]) - 1)) * (t[i] - n*self._T_symb)/self._T_symb

            phases[i] = phi_t
            result[i] = np.sqrt(2*self._E/self._T_symb)*np.exp(1j*phi_t)

        return result, t, phases
```

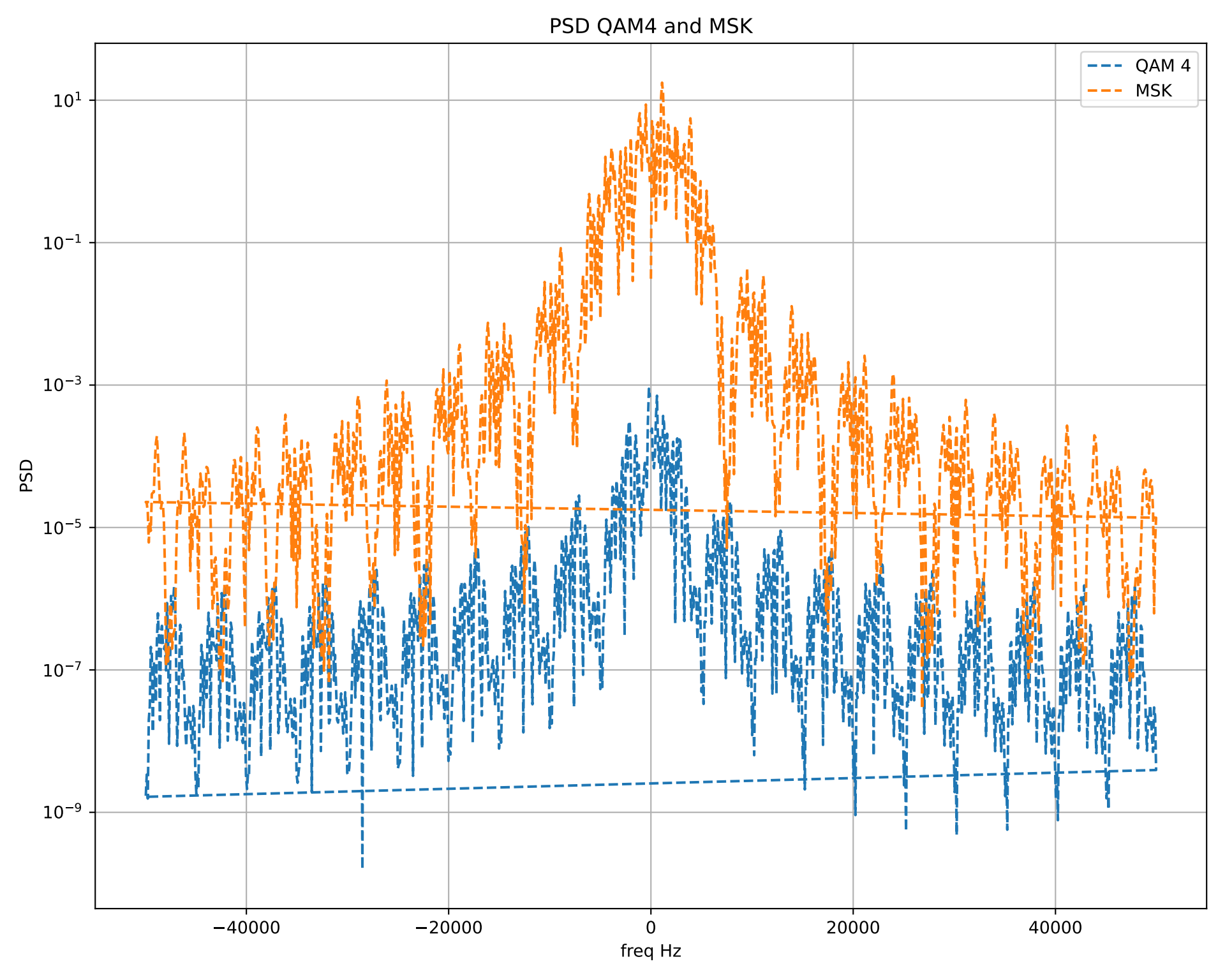

Amplitudes of QAM
Amp
1.4
1.2
1.0
0.8
0.6
0.4
0.000
0.002
0.004
0.006
0.008
0.010
time sec
qam 4
qam 16
qam 64
qam 256
Phases of QAM
Radians
3
2
1
0
−1
−2
−3
0.000
0.002
0.004
0.006
0.008
0.010
time sec
qam 4
qam 16
qam 64
qam 256

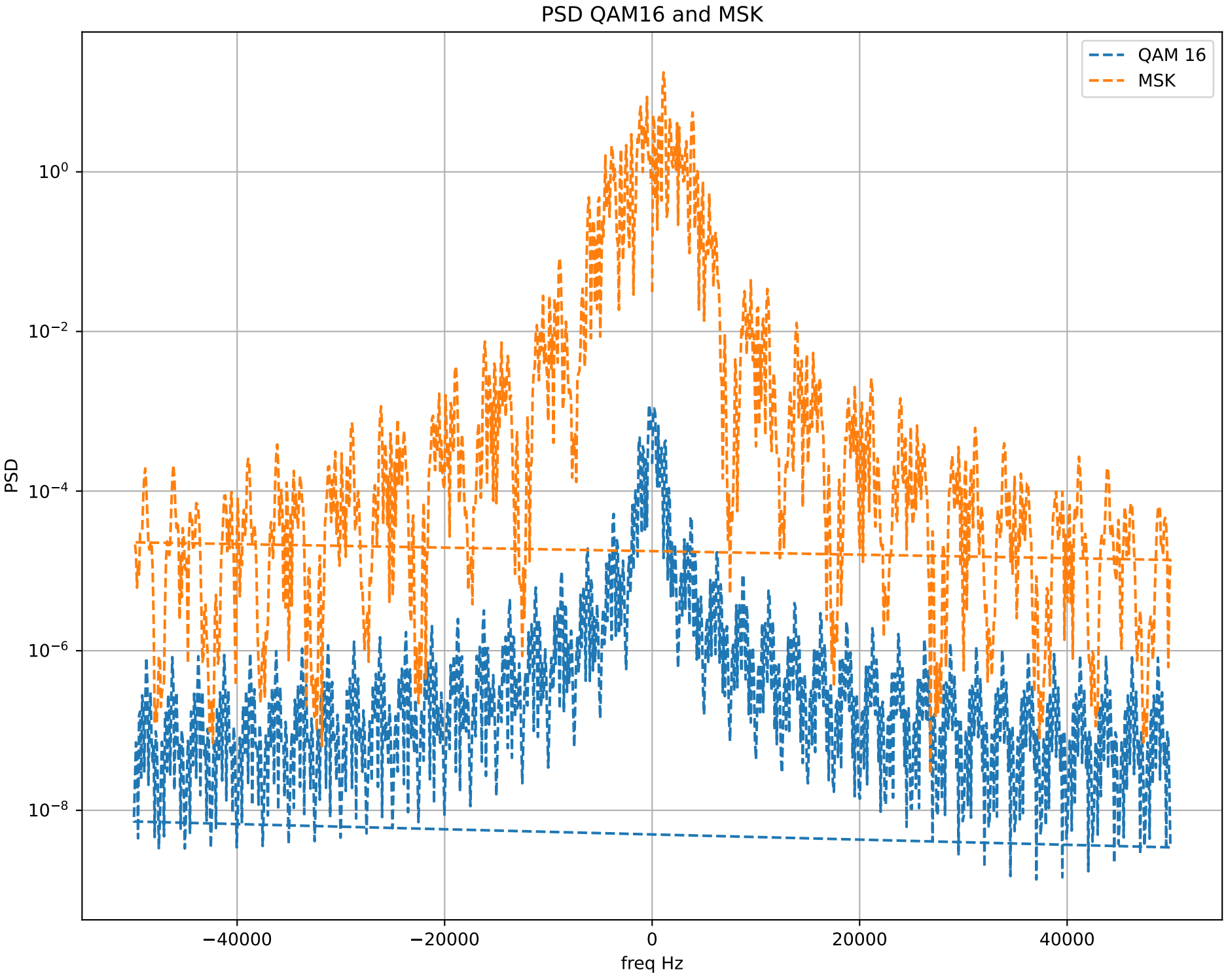

Baseband amplitudes of MSK
msk real
msk imag
Amp
time sec
BasebansPhases of MSK
msk
Radians
time sec

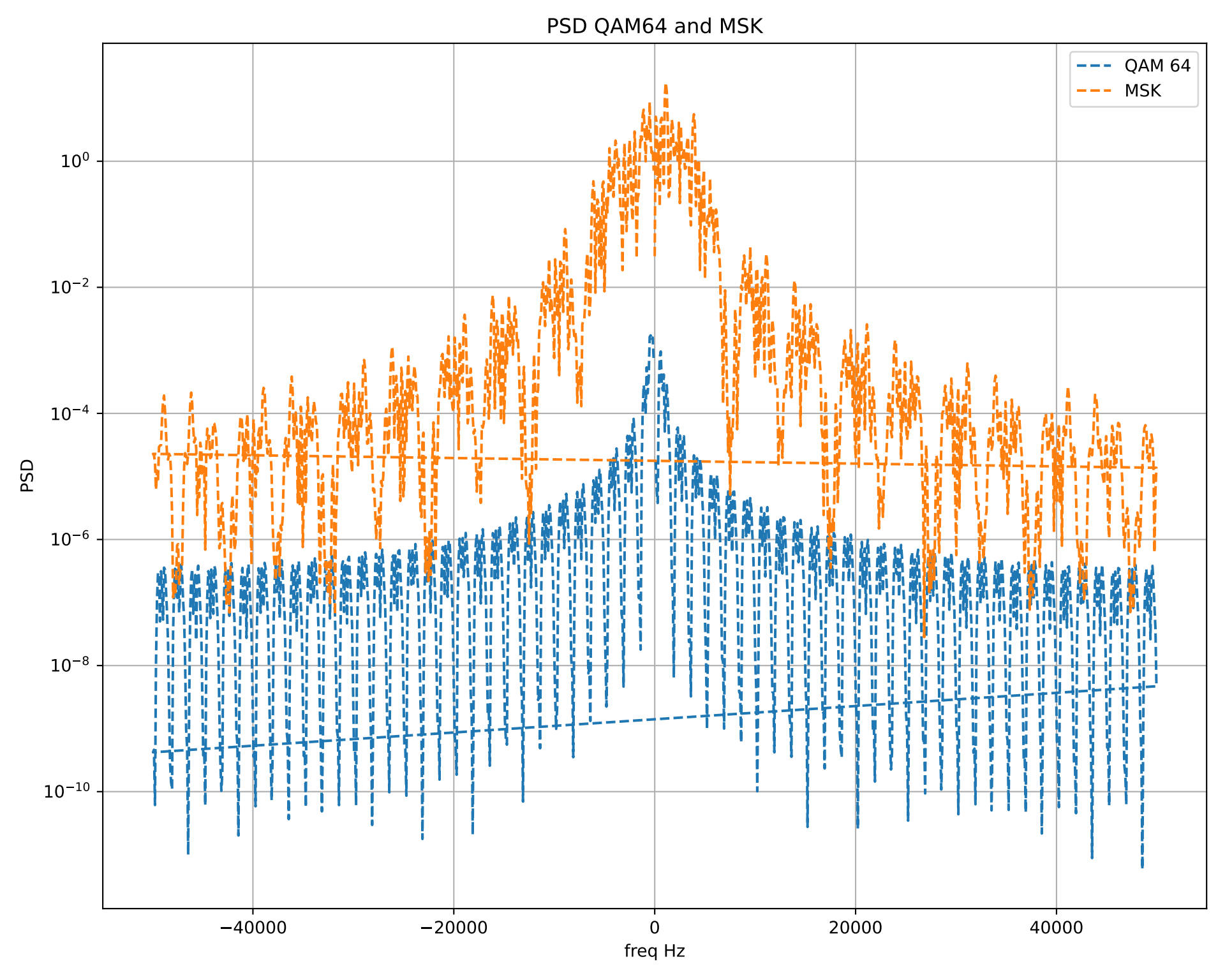

Amplitudes of QAM4
Amp
0.0
0.1
0.2
−40000
−20000
0
20000
40000
freq Hz
right spectral edge
left spectral edge
Amplitudes of QAM16
Amp
0.0
0.1
0.2
0.3
−40000
−20000
0
20000
40000
freq Hz
right spectral edge
left spectral edge
Amplitudes of QAM64
Amp
0.0
0.2
0.4
−40000
−20000
0
20000
40000
freq Hz
right spectral edge
left spectral edge
Amplitudes of QAM256
Amp
0.0
0.2
0.4
−40000
−20000
0
20000
40000
freq Hz
right spectral edge
left spectral edge

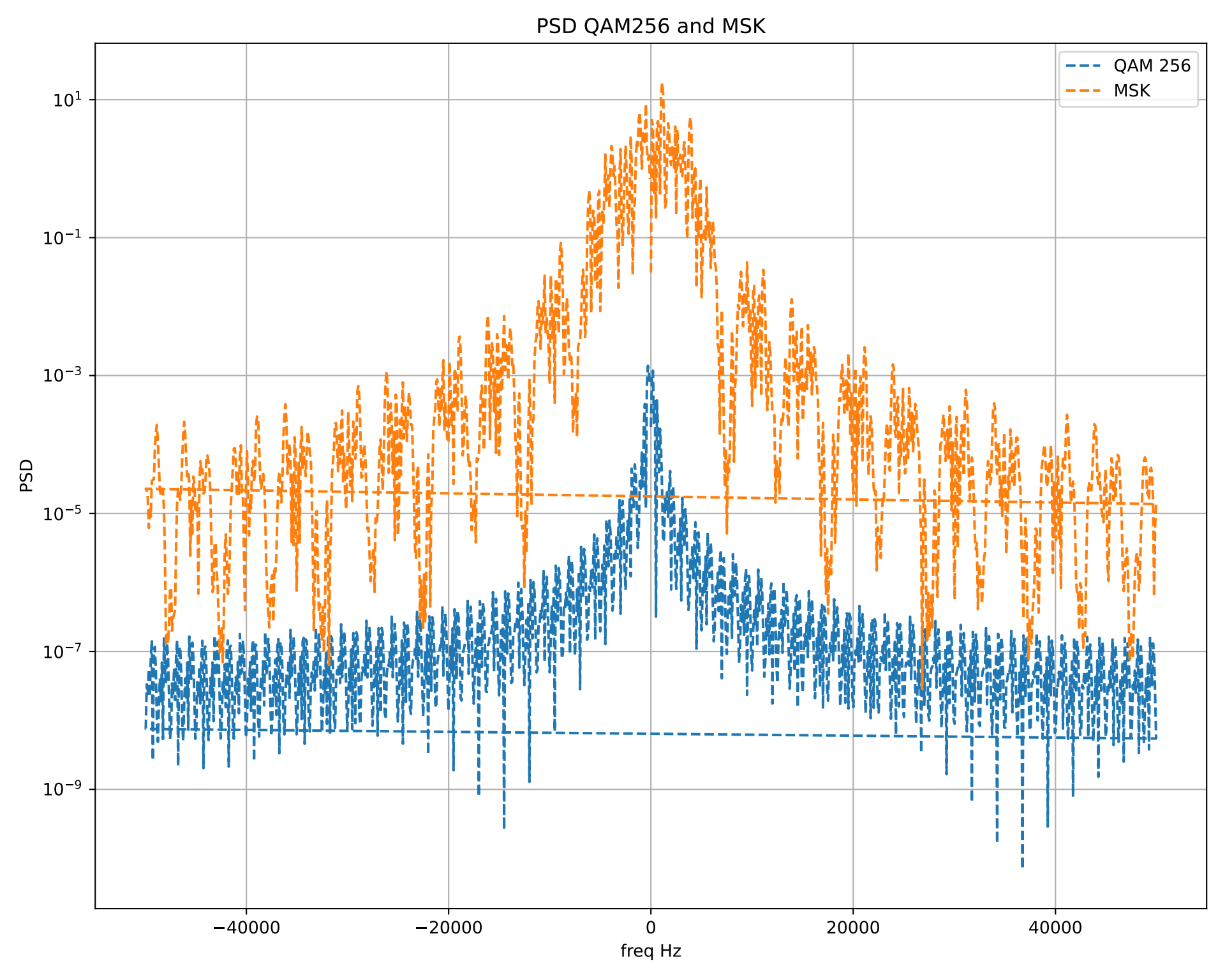

MSK spectrum's amplitude
right spectral edge
left spectral edge
Amp
freq Hz
MSK spectrum's phase
Radians
freq Hz

```
Bandwidth efficientivity:
BW_E QAM4= 0.15857142857142859   BW_E QAM4 teoreical= 2.0
BW_E QAM16= 0.26151832460732999   BW_E QAM16 teoreical= 4.0
BW_E QAM64= 0.28220338983050847   BW_E QAM64 teoreical= 6.0
BW_E QAM256= 0.5096938775510205   BW_E QAM256 teoreical= 8.0
BW_E MSK= 0.8325   BW_E MSK teoreical= 1.0
```


PSD QAM4 and MSK
QAM 4
MSK
PSD
freq Hz
10^{1}
10^{-1}
10^{-3}
10^{-5}
10^{-7}
10^{-9}
−40000
−20000
0
20000
40000


PSD QAM16 and MSK
QAM 16
MSK
10^{0}
10^{-2}
10^{-4}
10^{-6}
10^{-8}
PSD
−40000
−20000
0
20000
40000
freq Hz


PSD QAM64 and MSK
QAM 64
MSK
PSD
freq Hz
10^{0}
10^{-2}
10^{-4}
10^{-6}
10^{-8}
10^{-10}
−40000
−20000
0
20000
40000


PSD QAM256 and MSK
QAM 256
MSK
PSD
freq Hz
10^{1}
10^{-1}
10^{-3}
10^{-5}
10^{-7}
10^{-9}
−40000
−20000
0
20000
40000